

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Алтайский государственный технический университет им. И.И. Ползунова»
Университетский технологический колледж
Кафедра Информационные системы в экономике

Отчет защищен с оценкой _____

Преподаватель _____ С.В. Умбетов

« ____ » _____ 2026 г.

Отчет по лабораторной работе №2
по дисциплине «Инженерно-техническая поддержка
сопровождения интеллектуальных информационных систем»
«Основы работы с GIT»
ЛР 09.02.08.03.002

Студент группы	<u>1ИИС-41</u>	<u>Р.Н. Анненко</u>
	группа	и.о., фамилия
Преподаватель	<u>старший преподаватель, доцент</u>	<u>С.В. Умбетов</u>
	должность, ученая степень	и.о., фамилия

Барнаул 2026

Лабораторная работа №2

Основы работы с GIT

Цели и задачи работы: освоить базовые и продвинутые приёмы работы с распределённой системой контроля версий Git в операционной системе Linux. Сформировать практические навыки организации версионного контроля при разработке программного обеспечения и понимание роли Git в командной работе.

- изучить процесс установки и настройки Git;
- освоить основные команды для работы с локальным репозиторием;
- научиться управлять состояниями файлов, работать с ветками, выполнять слияния и разрешать конфликты.

Задание к работе: выполнить задания 1–20 в соответствии с условиями. Ответить на контрольные вопросы.



Задание принял: _____ Анненко Р.Н.

Подпись

Ход работы

Задание 1. Откройте терминал и проверьте наличие Git в системе командой `git --version`.

```
annenko_1iis41@ubuntuuserver:~$ git --version
git version 2.53.0
annenko_1iis41@ubuntuuserver:~$ _
```

Рисунок 1 – проверка наличия и версии GIT в системе

GIT изначально был установлен на моей версии ubuntu server

Задание 2. Определите свой дистрибутив Linux командой `cat /etc/os-release`.

```
annenko_1iis41@ubuntuuserver:~$ cat /etc/os-release
PRETTY_NAME="Ubuntu 26.04.1 LTS"
NAME="Ubuntu"
VERSION_ID="26.04"
VERSION="26.04.1 LTS (Resolute Raccoon)"
VERSION_CODENAME=resolute
ID=ubuntu
ID_LIKE=debian
HOME_URL="https://www.ubuntu.com/"
SUPPORT_URL="https://help.ubuntu.com/"
BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/"
PRIVACY_POLICY_URL="https://www.ubuntu.com/legal/terms-and-policies/privacy-policy"
UBUNTU_CODENAME=resolute
LOGO=ubuntu-logo
annenko_1iis41@ubuntuuserver:~$ _
```

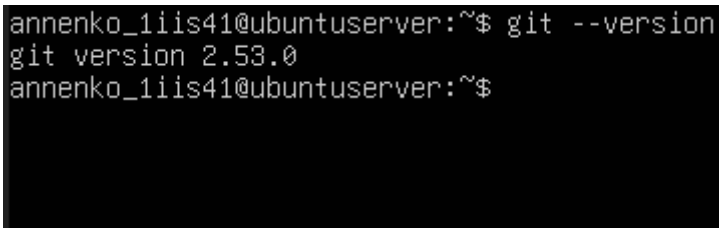
Рисунок 2 – информация о дистрибутиве linux

Задание 3. Установите Git с помощью менеджера пакетов вашего дистрибутива (`sudo apt-get install git` для Ubuntu/Debian, `sudo dnf install git` для Fedora, `sudo yum install git` для CentOS/RHEL, `sudo pacman -S git` для Arch Linux).

```
annenko_1iis41@ubuntuuserver:~$ sudo apt-get install git
[sudo: authenticate] Password:
sudo: Authentication failed, try again.
[sudo: authenticate] Password:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
git is already the newest version (1:2.53.0-1ubuntu1).
Solving dependencies... Done
The following packages were automatically installed and are no longer required:
  linux-headers-7.0.0-30 linux-headers-7.0.0-30-generic linux-image-unsigned-7.0.0-30-generic linux-main-modules-zfs-7.0.0-30-generic
  linux-modules-7.0.0-30-generic linux-tools-7.0.0-30 linux-tools-7.0.0-30-generic
Use 'sudo apt autoremove' to remove them.
0 upgraded, 0 newly installed, 0 to remove and 13 not upgraded.
annenko_1iis41@ubuntuuserver:~$ _
```

Рисунок 3 – установка GIT

Задание 4. Повторно проверьте версию Git командой `git --version` для подтверждения успешной установки.



```
annenko_1iis41@ubuntuserver:~$ git --version
git version 2.53.0
annenko_1iis41@ubuntuserver:~$
```

Рисунок 4 – повтор проверки версии GIT

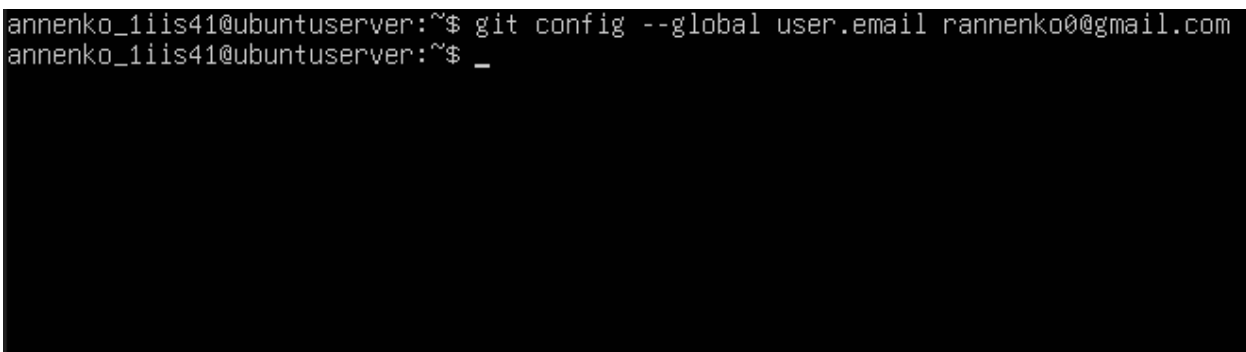
Задание 5. Настройте имя пользователя командой `git config -- global user.name "Ваше Имя"`.



```
annenko_1iis41@ubuntuserver:~$ git config --global user.name "Roman"
annenko_1iis41@ubuntuserver:~$
```

Рисунок 5 – настройка имени пользователя

Задание 6. Настройте адрес электронной почты командой `git config -- global user.email your.email@example.com`.



```
annenko_1iis41@ubuntuserver:~$ git config --global user.email rannenko0@gmail.com
annenko_1iis41@ubuntuserver:~$ _
```

Рисунок 6 – настройка адреса электронной почты

Задание 7. Проверьте применённые настройки командой `git config --list`.

```
annenko_1iis41@ubuntuserver:~$ git config --list
user.name=Roman
user.email=rannenko0@gmail.com
annenko_1iis41@ubuntuserver:~$ _
```

Рисунок 7 – Проверка настроек

Задание 8. Создайте рабочую директорию командой `mkdir git-practice` и перейдите в неё командой `cd git-practice`.

```
annenko_1iis41@ubuntuserver:~$ mkdir git-practice
annenko_1iis41@ubuntuserver:~$ cd git-practice
annenko_1iis41@ubuntuserver:~/git-practice$
```

Рисунок 8 – создание директории и переход в неё

Задание 9. Инициализируйте локальный репозиторий командой `git init`.

```
annenko_1iis41@ubuntuserver:~/git-practice$ git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: will change to 'main' in Git 3.0. To configure the initial branch name
hint: to use in all of your new repositories, which will suppress this warning,
hint: call:
hint:   git config --global init.defaultBranch <name>
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:   git branch -m <name>
hint: Disable this message with "git config set advice.defaultBranchName false"
Initialized empty Git repository in /home/annenko_1iis41/git-practice/.git/
annenko_1iis41@ubuntuserver:~/git-practice$
```

Рисунок 9 – инициализация локального репозитория

Задание 10. Убедитесь в создании скрытой папки .git командой ls -la.

```
annenکو_1iis41@ubuntuserver:~/git-practice$ ls -la
total 12
drwxrwxr-x 3 annenko_1iis41 annenko_1iis41 4096 Oct  5 10:54 .
drwxr-x--- 7 annenko_1iis41 annenko_1iis41 4096 Oct  5 10:53 ..
drwxrwxr-x 6 annenko_1iis41 annenko_1iis41 4096 Oct  5 10:54 .git
annenکو_1iis41@ubuntuserver:~/git-practice$ _
```

Рисунок 10 – проверка наличия папки .git

Задание 11. Создайте несколько файлов, например readme.txt, script.sh и data.json, с помощью команды touch.

```
annenکو_1iis41@ubuntuserver:~/git-practice$ touch readme.txt
annenکو_1iis41@ubuntuserver:~/git-practice$ touch script.sh
annenکو_1iis41@ubuntuserver:~/git-practice$ touch data.json
annenکو_1iis41@ubuntuserver:~/git-practice$ _
```

Рисунок 11 – создание файлов .txt .sh .json

Задание 12. Проверьте статус репозитория командой git status и определите состояние созданных файлов (untracked).

```
annenکو_1iis41@ubuntuserver:~/git-practice$ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        data.json
        readme.txt
        script.sh

nothing added to commit but untracked files present (use "git add" to track)
annenکو_1iis41@ubuntuserver:~/git-practice$ _
```

Рисунок 12 – проверка статуса репозитория

Задание 13. Добавьте в состояние staged только текстовые файлы командой `git add *.txt`.

```
annenko_1iis41@ubuntuserver:~/git-practice$ git add readme.txt
annenko_1iis41@ubuntuserver:~/git-practice$
```

Рисунок 13 – добавление в состояние staged текстового файла

Задание 14. Добавьте все остальные файлы командой `git add -A` и повторно проверьте статус командой `git status`.

```
annenko_1iis41@ubuntuserver:~/git-practice$ git add -A
annenko_1iis41@ubuntuserver:~/git-practice$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   data.json
        new file:   readme.txt
        new file:   script.sh

annenko_1iis41@ubuntuserver:~/git-practice$ _
```

Рисунок 14 – добавление остальных файлов и проверка статуса

Задание 15. Создайте первый коммит командой `git commit -m "initial commit: added base files"`.

```
annenko_1iis41@ubuntuserver:~/git-practice$ git commit -m "initial commit: added base files"
[master (root-commit) 6895f5b] initial commit: added base files
3 files changed, 0 insertions(+), 0 deletions(-)
create mode 100644 data.json
create mode 100644 readme.txt
create mode 100644 script.sh
annenko_1iis41@ubuntuserver:~/git-practice$
```

Рисунок 15 – создание коммита

Задание 16. Просмотрите историю коммитов командой `git log` и зафиксируйте метаданные коммита.

```
annenko_1iis41@ubuntuuserver:~/git-practice$ git log
commit 6895f5b95e98413963577c2ec2810285edc427c2 (HEAD -> master)
Author: Roman <rannenko0@gmail.com>
Date:   Mon Oct 5 11:01:52 2026 +0000

    initial commit: added base files
annenko_1iis41@ubuntuuserver:~/git-practice$
```

Рисунок 16 – просмотр истории коммитов

Задание 17. Внесите изменения в файл `readme.txt` и просмотрите различия командой `git diff`.

```
annenko_1iis41@ubuntuuserver:~/git-practice$ git diff
diff --git a/readme.txt b/readme.txt
index e69de29..5b2c740 100644
--- a/readme.txt
+++ b/readme.txt
@@ -0,0 +1 @@
+In case of fire: git commit, git push, leave the building.
annenko_1iis41@ubuntuuserver:~/git-practice$ _
```

Рисунок 17 – изменения файла `readme.txt` и проверка различий

Задание 18. Создайте файл `.gitignore` командой `touch .gitignore` и внесите в него шаблоны `*.log`, `temp/`, `*.tmp`, `.cache/`.

```
annenko_1iis41@ubuntuuserver:~/git-practice$ printf "*.log\n temp/\n *.tmp\n .cache/\n" > .gitignore
annenko_1iis41@ubuntuuserver:~/git-practice$ cat .gitignore
*.log
temp/
*.tmp
.cache/
annenko_1iis41@ubuntuuserver:~/git-practice$ git add .gitignore
annenko_1iis41@ubuntuuserver:~/git-practice$ git commit -m "added .gitignore file"
[master 29f392e] added .gitignore file
1 file changed, 4 insertions(+)
create mode 100644 .gitignore
annenko_1iis41@ubuntuuserver:~/git-practice$ _
```

Рисунок 18 – создание файла `.gitignore` и внесение в него шаблона

Задание 19. Создайте новую ветку командой `git checkout -b feature/update-readme` и проверьте список веток командой `git branch -a`.

```
annenko_1iis41@ubuntuserver:~/git-practice$ git checkout -b feature/update-readme
Switched to a new branch 'feature/update-readme'
annenko_1iis41@ubuntuserver:~/git-practice$ git branch -a
* feature/update-readme
  master
annenko_1iis41@ubuntuserver:~/git-practice$ _
```

Рисунок 19 – создание новой ветки и проверка списка веток

Задание 20. Внесите изменения в файл `readme.txt` в новой ветке, закоммитьте их, переключитесь на `master` командой `git checkout master`, выполните слияние командой `git merge feature/update-readme` и проанализируйте итоговую историю командой `git log --oneline --graph --all`.

```
annenko_1iis41@ubuntuserver:~/git-practice$ git add readme.txt
annenko_1iis41@ubuntuserver:~/git-practice$ git commit -m "added to readme"
[feature/update-readme 489ff19] added to readme
 1 file changed, 2 insertions(+)
annenko_1iis41@ubuntuserver:~/git-practice$ git checkout master
Switched to branch 'master'
annenko_1iis41@ubuntuserver:~/git-practice$ git merge feature/update-readme
Updating 29f392e..489ff19
Fast-forward
 readme.txt | 2 ++
 1 file changed, 2 insertions(+)
annenko_1iis41@ubuntuserver:~/git-practice$ git log --oneline --graph --all
* 489ff19 (HEAD -> master, feature/update-readme) added to readme
* 29f392e added .gitignore file
* 6895f5b initial commit: added base files
annenko_1iis41@ubuntuserver:~/git-practice$
```

Рисунок 20 – изменение файла `readme.txt` в новой ветке, коммит, переход в ветку `master`, слияние двух веток и финальные логи

Контрольные вопросы

1. Что такое Git и для чего он используется?

GIT – распределенная система контроля версий. Она используется, чтобы отслеживать историю изменений в файлах проекта, возвращаться к старым версиям кода и вести совместную разработку без риска перезаписать чужую работу.

2. В чём разница между локальным и удалённым репозиторием?

Локальный репозиторий находится прямо на компьютере в скрытой папке проекта, и можно работать с ним полностью автономно. Удалённый репозиторий размещается на сервере в интернете, например на GitHub, и служит центральным хабом для обмена кодом с командой и резервного копирования.

3. Какая команда проверяет наличие Git в системе и его версию?

Для проверки установленной версии используется команда `git --version`.

4. Какие команды используются для установки Git в Ubuntu, Fedora и Arch Linux?

В Ubuntu для установки используется команда `sudo apt update` и `sudo apt-get install git`. В дистрибутиве Fedora применяется команда `sudo dnf install git`. В системе Arch Linux установка выполняется через команду `sudo pacman -S git`.

5. Для чего нужна настройка `user.name` и `user.email` в Git?

Эти настройки связывают имя и почту с каждым сделанным коммитом. Это необходимо, чтобы другие разработчики понимали, кто именно автор конкретных изменений в коде.

6. В чём разница между командами `git config --global` и `git config` без флага?

Флаг `--global` применяет настройки для всех репозиториев текущего пользователя на этом компьютере. Команда без флага сохраняет настройки локально, поэтому они действуют только для одного конкретного проекта, в папке которого находишься.

7. Что создаётся в директории при выполнении команды `git init`?

При выполнении этой команды создаётся скрытая папка с именем `.git`, которая превращает обычную директорию в рабочий Git-репозиторий.

8. Для чего предназначена папка .git и можно ли её удалять?

В папке .git хранится вся служебная информация, история изменений, коммиты, ветки и конфигурация проекта. Удалять её нельзя, иначе проект перестанет быть репозиторием и вся история версий потеряется.

9. Какие четыре состояния файлов существуют в Git?

Файлы проходят четыре стадии. Сначала они новые и не отслеживаются системой (Untracked). Затем они становятся отслеживаемыми и чистыми (Unmodified). Если их отредактировать, они переходят в стадию измененных (Modified). После добавления в очередь на сохранение файлы становятся подготовленными (Staged), а после записи в историю – сохраненными (Committed).

10. В чём разница между состояниями untracked и modified?

Файл в состоянии Untracked является совершенно новым для репозитория, Git видит его в папке впервые и еще не отслеживает. Файл в состоянии Modified уже сохранен в истории контроля версий, но его старое содержимое было отредактировано.

11. Чем отличается команда git add . от git add -A и git add ?

Команда с указанием имени файла добавляет в индекс только один этот файл. Команда с точкой добавляет все новые, измененные и удаленные файлы из текущей папки и её подпапок. Команда с флагом -A добавляет вообще все изменения во всем репозитории, независимо от текущей папки.

12. Что такое коммит и какие метаданные он содержит?

Коммит – это сохранение (снимок) кода в определенный момент времени. В нем хранятся сами измененные файлы, имя автора, дата сохранения и комментарий к работе.

13. Для чего нужна команда `git status` и какую информацию она отображает?

Команда показывает текущее состояние рабочей директории. Она отображает имя активной ветки, информацию о синхронизации с сервером, а также списки измененных, подготовленных к коммиту или неотслеживаемых файлов.

14. Что показывает команда `git diff` и в каких случаях она применяется?

Она показывает построчную разницу между изменениями в файлах, выделяя добавленный и удаленный код. Применяется перед индексацией или коммитом, чтобы детально проверить, какие именно правки были внесены.

15. Для чего используется файл `.gitignore` и как формируются его шаблоны?

Он используется для указания файлов и папок, которые Git должен игнорировать и не добавлять в репозиторий, например логи или кэш. Шаблоны формируются по правилам регулярных выражений, где звездочка заменяет любые символы, а косой слэш на конце указывает на целую папку.

16. Что такое ветка в Git и почему её называют подвижным указателем?

Ветка – это независимая линия разработки. Технически это подвижный указатель, потому что ветка ссылается на самый последний сделанный в ней коммит и автоматически передвигается вперед при создании каждого нового коммита.

17. В чём разница между командами `git checkout master` и `git checkout -b new-branch`?

Команда `git checkout master` просто переключает вас на уже существующую ветку `master`. Команда `git checkout -b new-branch` одновременно создает новую ветку с указанным именем и сразу же переключает на неё.

18. Что происходит при выполнении команды `git merge` и в каком случае возникает конфликт?

Команда `git merge` сливает изменения из указанной ветки в текущую ветку. Конфликт возникает, если в обеих ветках одновременно были изменены одни и те же строчки в одном и том же файле, из-за чего Git не может сам решить, какую версию кода оставить.

19. Как разрешить конфликт слияния и какие маркеры при этом используются в файле?

Для разрешения конфликта нужно открыть проблемный файл, вручную выбрать правильный вариант кода, удалить технические маркеры, сохранить файл и сделать новый коммит. В файле используются маркеры HEAD для начала изменений, семь знаков равно в качестве разделителя версий и маркер с именем ветки для обозначения конца входящих изменений.

20. Для чего применяется команда `git log` и что означает флаг `--oneline --graph --all`?

Команда `git log` выводит историю всех сделанных коммитов репозитория. Флаг `--oneline` сжимает каждый коммит в одну короткую строку, флаг `--graph` рисует текстовое дерево ветвлений, а флаг `--all` показывает историю абсолютно всех веток проекта.

Вывод

В ходе лабораторной работе я проверил наличие **Git** командой `git --version`, задал глобальные настройки через `git config --global` и проверил их с помощью `git config --list`. Далее я создал директорию и инициализировал в ней репозиторий командой `git init`, проверил статус файлов через `git status`, добавил их в индекс с помощью `git add` и создал первый коммит командой `git commit`, зафиксировав изменения в истории `git log`. Затем я изучил отличия в файлах с помощью `git diff`, настроил исключения в `.gitignore`, создал новую ветку командой `git checkout -b` и проверил её наличие через `git branch -a`. В финале я вернулся в главную ветку с помощью `git checkout master`, выполнил слияние командой `git merge` и успешно проверил итоговую структуру репозитория через `git log --oneline --graph --all`. В результате я получил практические навыки работы с распределённой системой контроля версий **Git** в операционной системе **Linux**, освоил управление состояниями файлов, работу с ветками и выполнение слияний.

[Мой репозиторий на github](#)